relating `Model` instance A to `Model` instance B would not add another row for relating B back to A.

The table used to store a many-to-many relationship is automatically generated by Django. You can stop this and specify a different `Model` object to customize the relationship, using the `through` argument. This is normally done to add extra fields associated with the relationship. When doing so, the `through_fields` argument specifies names of the fields that form the relationship. This is only necessary if there are multiple `ForeignKey` fields pointing to the same `Model`, making the relationship ambiguous.

D.3.3 OneToOneField

```
class OneToOneField(to, on_delete, parent_link=False, **options)
```

This is similar to a `ForeignKey`, but it enforces a unique relationship constraint. A related `Model` can only be connected to a single instance of the owning object. The `to` and `on_delete` arguments are the same as `ForeignKey`.

The `parent_link` argument can only be used if the `Model` inherits from another concrete model. In this case, if `parent_link` is `True`, the field relates the parent class rather than the otherwise automatically created field. For more information on model inheritence, see https://docs.djangoproject.com/en/dev/topics/db/models/#model-inheritance.